

CRM Integration for the Onboarding Engine

Technical interface specification for automated onboarding of new real estate agency clients

PROJECT	UPGRADS Real Estate Agent System · Automated Client Onboarding
VERSION	01 September 2026
RECIPIENT	CRM Development

INTERFACE PRINCIPLE

The CRM remains the system of record for master data. The engine only writes back content that it has generated itself.

1. Purpose

We are automating the onboarding of new real estate agency clients. Today, everything after contract signature is handled manually: creating folders, sending the form, coordinating an appointment, and gathering information. Going forward, an engine will take over these tasks.

The engine is a standalone service outside the CRM. It needs two things from the CRM: notification when something happens, and access to what we already know about the client.

Your responsibility is the interface. We will build the engine itself.

2. Process

1. A lead comes in. The CRM creates the client. The engine creates a client folder in Google Drive and a client record.
2. Sales call. The call is recorded. The engine evaluates the transcript and populates fields in the record with the status inferred.
3. Contract signed. This triggers onboarding.
4. Onboarding form. The form is sent to the client, prefilled with everything we already know. The client reviews it, adds missing information, and uploads materials.
5. Appointment booking. Booking takes place directly on the form confirmation page.
6. Onboarding call. The recording is evaluated. The engine creates `briefing.md` and `offene-fragen.md`, then reports back.

3. System Responsibilities

System	Responsible for
CRM (your part)	Master data, status, communication, documents, triggers
Engine (our part)	Evaluation, form, folder logic, document generation
Google Drive	Storage: assets, recordings, transcripts, results

The CRM remains authoritative for master data. The engine only writes back content that it has generated itself.

4. Core Concept: Field Status

Every information field about a client has one of three states:

- `bestaetigt` - obtained from the contract, legal notice, or a direct statement by the client. It is not requested again.
- `vermutet` - inferred from a conversation or email exchange. Always includes a source and supporting passage. It is presented for verification rather than requested again.
- `offen` - not answered anywhere.

Hard rule: Information without evidence must not be added. No plausible gap-filling. A field without a source is `offen`, even if the answer appears obvious.

This is not a minor detail; it determines whether the system works. Otherwise, fabricated content could end up on the client's website without review.

5. What You Need to Build

5.1 Webhooks: CRM Notifies the Engine of Events

Outgoing webhooks using HTTP POST and JSON. We will provide the target URL; it must be configurable.

Events

Event	When	Purpose
kunde.angelegt	New lead in the CRM	Create client folder and record
kunde.aktualisiert	Master data changed	Update the record
vertrag.unterschrieben	Signed contract is available	Start onboarding
dokument.hinzugefuegt	New document attached to the client	Evaluate it, e.g. contract or recording

Example: `vertrag.unterschrieben`

```
{
  "event": "vertrag.unterschrieben",
  "event_id": "evt_01J8X2K9",
  "zeitpunkt": "2026-09-01T14:23:11Z",
  "kunde_id": "kd_7f3a91",
  "daten": {
    "vertrag_id": "vtr_2201",
    "produkt": "maklersystem_landingpage",
    "unterschrieben_am": "2026-09-01",
    "laufzeit_monate": 12,
    "dokument_url": "https://crm.upgrads.de/api/dokumente/dok_5512"
  }
}
```

Requirements

- Signature: HMAC-SHA256 over the raw request body using a shared secret. Header: X-Upgrads-Signature.
- `event_id`: unique and stable. We process events idempotently; duplicate delivery must not cause damage.
- Retries on error: at least five attempts with increasing intervals, extending to approximately 24 hours. We respond with HTTP 200 upon acceptance.
- Delivery log: visible in the CRM so delivery issues can be investigated.

5.2 REST Endpoints: Engine Reads and Writes

Authentication via API key in the header or OAuth 2.0 Client Credentials. Your choice; just tell us which option you implement.

Read

```
GET /api/kunden/{kunde_id}
-> Master data: company name, legal form, address, contact person,
phone, email, VAT ID, registration number, website, source, status

GET /api/kunden/{kunde_id}/kommunikation?typ=email&seit=2026-01-01
-> Email correspondence: subject, body, direction, timestamp, sender

GET /api/kunden/{kunde_id}/dokumente
-> List: id, typ, dateiname, mimetype, groesse, erstellt_am, download_url

GET /api/dokumente/{dokument_id}/download
-> Binary data; a time-limited link is acceptable

GET /api/kunden/{kunde_id}/notizen
-> Sales call notes
```

Write

```
PATCH /api/kunden/{kunde_id}
-> Add master data identified by the engine
(e.g. legal-notice data, registration number from research)

POST /api/kunden/{kunde_id}/notizen
-> Engine result logs as a note

PATCH /api/kunden/{kunde_id}/status
-> Set onboarding status (see 5.3)
```

Important when writing: The engine never overwrites a field already populated in the CRM. It only fills empty fields. If a value differs, it writes a note instead of overwriting the field. Please enforce this behavior at the endpoint level.

5.3 Onboarding Status Values

A field on the client record maintained by the engine:

```
kein_onboarding Default; no contract yet
formular_versendet Contract signed; form sent
formular_ausgefuehlt Client submitted the form
termin_gebucht Onboarding appointment scheduled
call_erfolgt Recording is available
briefing_fertig briefing.md created; open questions sent
onboarding_abgeschlossen All open questions resolved
```

5.4 Storing the Linkage

We need two additional fields on the client record:

- `drive_ordner_id` - Google Drive folder ID of the client's folder
- `akte_version` - counter used to detect concurrent changes

6. Client Record Data Model

Recommendation: do not store the complete record in the CRM. It contains approximately 60 fields, each with a status, source, and supporting passage. Modeling this through CRM custom fields would make the CRM cluttered and would never remain maintainable.

The record is stored as `kunde.json` in the client's Drive folder. The CRM retains master data and status - the information already visible to the sales team.

If you disagree because the CRM should remain authoritative, we need a generic endpoint:

```
GET /api/kunden/{kunde_id}/akte
PUT /api/kunden/{kunde_id}/akte
```

This endpoint stores a JSON document without the CRM needing to understand its structure. This is the pragmatic option. Please provide your assessment.

Structure of a field in the record

```
{
  "positionierung_abgrenzung": {
    "wert": "Works exclusively with property owners; no buyer search. Justifies this with conflicts of interest.",
    "status": "vermutet",
    "quelle": "salesgesprach_2026-08-14",
    "beleg": "Minute 12:30, paraphrased: he did not want to represent both sides",
    "aktualisiert_am": "2026-08-14T16:02:00Z"
  }
}
```

7. Field Catalog

The engine works against a fixed catalog. Groups and typical sources:

Group	Examples	Typical source
Company data	Company name, legal form, address, contact	Contract, legal notice
Brand and design	Logo, colors, fonts, guidelines	Form (upload)
Positioning	Differentiation, self-description, values, claim	Sales call, onboarding call
Target audience	Owner types, property types, preferred properties	Sales call
Services	Scope of services, areas of focus	Sales call, form
Sales process	Process, steps, valuation offer	Onboarding call
Region	Locations, districts, local networks	Sales call, research
Trust	Metrics, reviews, reference properties	Form, research
Images	Team, office, properties, other media	Form (upload)
Legal	Legal notice, privacy, Section 34c permit	Contract, research
Leads	Target inbox, calendar, primary CTA	Form

We will provide the complete catalog with all field names later as a JSON Schema. For your work, the only relevant point is that the record is a freely structured JSON document.

8. Technical Requirements

- Authentication: API key or OAuth 2.0 Client Credentials - please propose your preferred option.
- Idempotency: every write request accepts an `Idempotency-Key` header.
- Rate limits: please specify them so we can comply.
- Errors: meaningful HTTP status codes and a `fehler` object containing `code` and `nachricht`. Do not return HTTP 200 with an error message in the response body.
- Timestamps: consistently use ISO 8601 in UTC.
- Test environment: a separate instance with test data before connecting to production.
- Documentation: an OpenAPI specification is fully sufficient.

9. Out of Scope

To avoid overlap, we will build the following; these are not CRM responsibilities:

- Onboarding form and its hosting
- Transcription and evaluation of recordings

- Google Drive folders and uploads
- Generation of `briefing.md` and `offene-fragen.md`
- Sending emails to the client
- Appointment booking

10. Open Decisions

- Record in the CRM or in Drive? See Section 6. We recommend Drive, but need your assessment.
- Contract trigger. Ideally, a signature tool reports when a contract is signed. If contracts continue to be returned by email, we need a manual trigger in the CRM: a "Start onboarding" button that fires the webhook. This is significantly more robust than monitoring an inbox.
- Calendar. Which tool will be used for appointment booking, and does the CRM have its own scheduling feature or should we connect an external tool?
- Recordings. Which platform is used for calls? If Google Meet or Zoom is used, the transcription step is eliminated entirely because both provide completed transcripts.

11. Delivery Sequence

We do not need everything at once. A sensible rollout is:

Stage 1 - enough to begin: `GET /api/kunden/{id}`, `webhook vertrag.unterschrieben`, `status` field

Stage 2: document access, `kunde.angelegt`, write endpoints

Stage 3: communication and notes for evaluating earlier conversations

With Stage 1, we can already test the complete onboarding flow end to end.